МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт № 8 «Компьютерные науки»

ЛАБОРАТОРНАЯ РАБОТА № 1

по курсу «Информационный поиск»

«Добыча корпуса документов»

Выполнил: Д. А. Кулешов
Группа: М8О-408Б-23
Преподаватель: А. А. Кухтичев
Отметка о сдаче: _____
Оценка: _____

Москва 2026

1 Задание

Цель работы — выбрать источники для будущего поискового индекса, получить примеры документов, изучить их формат и метаданные, выделить основной текст и оценить количественные характеристики выбранного корпуса.

В соответствии с заданием необходимо:

- использовать не менее двух источников документов;
- скачать примеры документов и указать источники данных;
- изучить состав сырого документа, его разметку и метаданные;
- выделить из сырого представления основной текст;
- найти существующие поисковые системы, работающие с выбранными источниками;
- выполнить несколько запросов и указать недостатки полученной выдачи;
- определить количество документов, общий и средний размер сырых документов, общий и средний размер выделенного текста.

В качестве предметной области я выбрал англоязычные технические тексты по компьютерным наукам и информационным технологиям. В корпус входят три источника: Wikipedia, arXiv HTML и Microsoft Learn. Wikipedia содержит энциклопедические статьи, arXiv — научные публикации, Microsoft Learn — прикладную техническую документацию. Таким образом, документы относятся к общей технической тематике, но отличаются жанром и структурой.

2 Краткое описание метода решения

Анализ каждого источника выполняется в отдельном Jupyter Notebook. Для получения документов используются официальные способы обнаружения данных: MediaWiki API, Atom API arXiv и sitemap Microsoft Learn. После загрузки я изучаю структуру сырого документа и метаданные, выделяю основной текст по правилам конкретного источника, измеряю размеры и вручную проверяю несколько страниц.

Для работы используются Python, `requests`, Beautiful Soup, `lxml`, `pandas` и `matplotlib`. Зависимости фиксируются проектом и устанавливаются через `uv`. Воспроизводимый запуск анализа выполняется командами:

```
uv sync
uv run jupyter notebook notebooks
```

В Jupyter необходимо последовательно выполнить `Run All` для файлов `wikipedia.ipynb`, `arxiv.ipynb` и `microsoft_learn.ipynb`. Все строки и файлы обрабатываются в UTF-8.

3 Журнал выполнения задания

3.1 Подготовка и порядок анализа

Для каждого источника я использовал одинаковую последовательность действий:

1. обнаружил адреса документов с помощью официального API или sitemap;
2. загрузил 30 примеров и сохранил их локально;
3. определил поля метаданных и область страницы с основным содержимым;
4. удалил служебные блоки и нормализовал пробельные символы;
5. измерил размер сырого представления и выделенного текста в байтах UTF-8;
6. подсчитал слова разделением текста по пробельным символам;
7. проверил существующий поиск и записал наблюдаемые ограничения.

Локальный кэш размещается в `notebooks/samples`. Он исключён из Git, так как содержит загруженные документы, но позволяет повторно выполнять анализ без новых запросов к внешним сайтам. Код и результаты анализа сохраняются непосредственно в ноутбуках.

Размер сырого документа вычисляется как длина его сохранённого представления в байтах UTF-8. Размер текста вычисляется после выделения основного содержимого и нормализации пробелов. Среднее значение равно сумме размеров, делённой на число документов. Для сравнения источников также рассчитана доля текста в сыром документе.

3.2 Wikipedia

Для Wikipedia использован официальный MediaWiki API:

https://en.wikipedia.org/w/api.php

В выборку включены страницы из категорий `Computer science`, `Engineering`, `Mathematics`, `Physics`, `Chemistry` и `Biology`. Сначала методом `categorymembers` получаются кандидаты, затем для каждой страницы запрашиваются идентификатор, заголовок, URL, Wikidata ID и поле `extract`.

Основные параметры запроса выглядят следующим образом:

```
response = session.get(API_URL, params={
    'action': 'query',
    'format': 'json',
    'formatversion': 2,
    'pageids': page_id,
    'prop': 'extracts|info|pageprops',
    'explaintext': 1,
    'inprop': 'url',
}, timeout=60)
```

Параметр `explaintext=1` просит MediaWiki вернуть обычный текст без HTML-разметки. Поэтому выделение текста для этого источника состоит в чтении поля `extract` и схлопывании повторных пробельных символов:

```
text = re.sub(r'\s+', ' ', page.get('extract', '')).strip()
```

Сырой документ в этом ноутбуке — JSON-объект API, а не HTML страницы. В нём сохраняются `pageid`, `title`, `fullurl`, `wikibase_item`, категория выборки и текстовый `extract`. Это важно учитывать при сравнении размеров: доля текста в JSON Wikipedia ожидаемо выше, чем доля текста в HTML двух других источников.

В качестве ручного примера проверена статья Computer science. Её `pageid` равен 5323, размер JSON в выборке — 30 814 байт, размер выделенного текста — 30 137 байт, число слов — 4 306. Заголовок, URL и Wikidata ID были получены отдельно от текста и не потерялись при нормализации.

Результат анализа Wikipedia: все 30 документов содержат непустой связный текст. Общий размер JSON-документов составил 643 895 байт, выделенного текста — 623 248 байт. Средний документ содержит 3 121,37 слова. Разница между сырьём и текстом небольшая, потому что MediaWiki API

уже выполнил удаление HTML-разметки. Источник подходит для корпуса, но при массовой загрузке HTML правила выделения текста придётся проверять отдельно.

3.3 arXiv HTML

Метаданные arXiv получены через Atom API:

https://export.arxiv.org/api/query

Для анализа выбраны по 10 публикаций из категорий `cs.IR`, `cs.AI` и `cs.CL`. Таким образом, выборка состоит из 30 документов и охватывает информационный поиск, искусственный интеллект и компьютерную лингвистику. Из Atom сохраняются идентификатор статьи, название, авторы, аннотация, категории, даты публикации и обновления, лицензия.

Сырой документ загружается с адреса `https://arxiv.org/html/<arxiv-id>`. Между HTML-запросами установлена пауза в одну секунду. При кодах 429 и 503 запрос повторяется после увеличенной паузы либо значения заголовка `Retry-After`.

Основное содержимое находится в `article.ltx_document`, `article` или `main`. После выбора корневого узла удаляются скрипты, стили, навигация, боковые панели и подвал. Текст собирается из заголовков, абзацев, элементов списков и блоков кода:

```
article = soup.select_one('article.ltx_document, article, main')
for node in article.select('script, style, nav, aside, footer'):
    node.decompose()
blocks = [
    node.get_text(' ', strip=True)
    for node in article.select('h1, h2, h3, p, li, pre')
]
text = re.sub(r'\s+', ' ', ' '.join(blocks)).strip()
```

HTML arXiv создаётся из LaTeX и содержит большое количество классов, формул и служебной разметки. Формулы переходят в линейное текстовое представление, поэтому часть двумерной структуры теряется. Это ограничение допустимо для полнотекстового поиска, но исходный HTML необходимо сохранять, чтобы в дальнейшем можно было улучшить правила обработки.

В качестве примера вручную разобрана статья Multimodal Reranking Is a System One Decision. Её HTML занимает 393 967 байт, выделенный текст — 36 089 байт, число слов — 5 806. В текст попали название, разделы и абзацы статьи, а навигация и оформление страницы были удалены.

Результат анализа arXiv: получено 30 непустых HTML-статей. Их общий сырой размер равен 16 108 664 байтам, а размер выделенного текста — 2 242 943 байтам. Средняя статья содержит 11 931,97 слова. Это самый длинный и самый сложный по разметке источник. Он даёт точную научную терминологию, но требует аккуратной обработки формул, библиографии и LaTeXML-разметки.

3.4 Microsoft Learn

Адреса страниц Microsoft Learn получены через индекс sitemap:

https://learn.microsoft.com/_sitemaps/sitemapindex.xml

Из sitemap выбираются только англоязычные адреса `/en-us/`. Исключаются ответы пользователей, видео, события, профили, страницы сертификатов, API-справочник и архив старых версий. Чтобы не получить 30 однотипных страниц одного продукта, документы поочерёдно выбираются из разделов Azure, SQL, Windows, Linux, Security, Power BI, DevOps, Cosmos DB, Semantic Kernel, C++, Defender и AI.

Основной материал страницы находится внутри `main`. Из него удаляются скрипты, стили, навигация, кнопки, уведомления, метаданные статьи, оглавление и форма обратной связи. Код и таблицы сохраняются, так как для технической документации они содержат значимые термины.

```
main = soup.select_one('main')
for node in main.select(
    'script, style, nav, aside, footer, button, .notification, '
    '#article-header, #article-metadata, '
    '#site-user-feedback-footer, .doc-outline'
):
    node.decompose()

blocks = [
    node.get_text(' ', strip=True)
    for node in main.select('h1, h2, h3, p, li, pre, table')
]
```

Из метатегов отдельно сохраняются `document_id`, описание, автор и дата. Из элемента `link rel="canonical"` извлекается канонический URL. Такое разделение позволяет сохранить метаданные для будущей фильтрации, но не добавлять их повторно в основной текст.

В качестве ручного примера проверена страница Index vector data in Cosmos DB. Её HTML занимает 61 456 байт, выделенный текст — 14 273 байта, число слов — 2 149. После очистки остались описание векторного индекса, параметры, списки, таблицы и примеры конфигурации.

Результат анализа Microsoft Learn: получено 30 документов из разных продуктовых разделов. Общий HTML занимает 1 770 272 байта, выделенный текст — 307 385 байт, среднее число слов равно 1 474,33. Основной текст заметно короче, чем у arXiv, однако в нём встречаются важные для поиска имена команд, классов, параметров и фрагменты кода. Удаление навигации и повторяющихся служебных блоков обязательно.

3.5 Сопоставление результатов анализа метаданных

Состав сохранённых метаданных приведён в таблице 1.

Таблица 1 — Метаданные исследованных источников

Источник	Основные поля	Назначение
Wikipedia	`pageid`, заголовок, URL, Wikidata ID, категория	идентификация статьи и связь с категорией
arXiv	arXiv ID, авторы, категории, даты, лицензия, аннотация	поиск научных работ и фильтрация по предметной области
Microsoft Learn	document ID, продукт, автор, дата, описание, canonical URL	идентификация версии документа и принадлежности продукту

Анализ показал, что во всех трёх источниках можно сохранить устойчивый идентификатор, заголовок и URL. Дополнительные поля различаются: Wikipedia даёт связь с Wikidata, arXiv — авторов, предметные категории и лицензию, Microsoft Learn — продукт, автора документа и дату. Эти поля не следует смешивать с основным текстом, но их нужно сохранить для фильтрации и будущего зонного поиска.

3.6 Возникшие проблемы и выбранные решения

Для каждого источника был создан отдельный ноутбук. Это упростило повторный запуск и позволило не смешивать разные правила получения и очистки документов.

При работе с Wikipedia выяснилось, что полный `extract` удобнее получать по одной странице. Сначала я загружаю небольшой список кандидатов из нескольких категорий, а затем последовательно запрашиваю содержимое. Пустые и слишком короткие статьи не включаются: минимальный объём установлен равным 200 словам.

Для arXiv основной проблемой стала стоимость повторной загрузки больших HTML- страниц и возможность временных ответов 429 или 503. Поэтому запросы выполняются последовательно, между ними есть пауза, а готовая выборка кэшируется. В выборку включаются только статьи, для которых доступна HTML-версия. Это создаёт некоторое смещение выборки, поскольку не все документы arXiv имеют HTML.

У Microsoft Learn один sitemap может содержать много однотипных страниц одного раздела. Простое чтение первых 30 адресов дало бы нерепрезентативную выборку. Поэтому я разделил адреса по продуктам и выбирал их поочерёдно. Дополнительно исключил страницы API и старых версий. После очистки проверяется минимальный объём в 250 слов.

Единый CSS-селектор для всех трёх источников использовать нельзя. Wikipedia в ноутбуке уже возвращает текст в JSON, arXiv хранит статью в `article`, а Microsoft Learn — в `main`. Я оставил отдельные функции выделения текста. Это немного увеличивает код, но делает правила понятными и не приводит к смешиванию основного содержимого с навигацией.

Размеры источников сначала выглядели несопоставимыми. Причина состоит в том, что у Wikipedia сырой документ — компактный JSON с уже очищенным `extract`, а у arXiv и Microsoft Learn — полный HTML. Поэтому в результатах я указываю не только абсолютные размеры, но и явно определяю, что именно считается сырым документом.

4 План тестирования

План составлен так, чтобы проверить загрузку, повторяемость, качество выделения текста и корректность статистики. Тесты выполняются отдельно для каждого ноутбука. План приведён в таблице 2.

Таблица 2 — План и результаты тестирования

№	Проверка	Ожидаемый результат	Результат
1	Установка через `uv sync`	Все зависимости устанавливаются без ручного подбора версий	Пройдено
2	Выполнение `Run All`	Ячейки выполняются последовательно, статистика выводится	Пройдено
3	Число документов	В каждом источнике получено ровно 30 документов	Пройдено
4	Уникальность идентификаторов	Внутри выборки нет повторяющихся page ID, arXiv ID и URL	Пройдено
5	Наличие текста	После очистки нет пустых документов	Пройдено
6	Порог длины	Wikipedia содержит не менее 200 слов, Learn — не менее 250 слов	Пройдено
7	Метаданные	У документов есть идентификатор, заголовок и URL	Пройдено
8	Размеры	Суммы размеров совпадают с суммой строк таблицы документов	Пройдено

Продолжение таблицы 2

№	Проверка	Ожидаемый результат	Результат
9	Ручная проверка текста	Для одной страницы каждого источника отсутствуют меню и служебные блоки	Пройдено
10	Повторный запуск	При наличии кэша используются те же документы без повторной загрузки	Пройдено
11	Существующий поиск	Ссылки на поиск открываются и возвращают результаты	Пройдено

Проверка уникальности выполняется до добавления документа в выборку. В Wikipedia используется множество `seen_page_ids`, в arXiv — `seen_ids`, в Microsoft Learn — `seen_urls`. Поэтому одна и та же страница не учитывается два раза в статистике.

Ручная проверка проводилась сравнением страницы в браузере с полем `text` в таблице ноутбука. Для Wikipedia проверена статья `Computer science`, для arXiv — `2610.28435v1`, для Microsoft Learn — страница об индексации векторов в Cosmos DB.

В этой работе не создаётся самостоятельная поисковая программа или интерфейс ввода запросов: результатом является воспроизводимый анализ источников в трёх ноутбуках. Поэтому требование о консольной утилите для поисковых лабораторных к данному этапу не применяется. Результаты каждого теста остаются в выходных ячейках ноутбуков.

5 Результаты

5.1 Количественные характеристики

Результаты измерений приведены в таблице 3. Размеры указаны в байтах UTF-8.

Таблица 3 — Размеры документов исследовательской выборки

Источник	Кол-во, шт.	Сырой размер, байт	Размер текста, байт	Средний сырой, байт/док.	Средний текст, байт/док.
Wikipedia	30	643 895	623 248	21 463,17	20 774,93
arXiv	30	16 108 664	2 242 943	536 955,47	74 764,77
Microsoft Learn	30	1 770 272	307 385	59 009,07	10 246,17
Всего	**90**	**18 522 831**	**3 173 576**	**205 809,23**	**35 261,96**

Среднее количество слов в документе равно 3 121,37 для Wikipedia, 11 931,97 для arXiv и 1 474,33 для Microsoft Learn. Наиболее длинными оказались научные статьи arXiv. Документация Microsoft Learn в среднем короче, но содержит больше кода, таблиц и узких названий параметров.

Доля выделенного текста в сыром представлении составляет 96,79 % для Wikipedia, 13,92 % для arXiv и 17,36 % для Microsoft Learn. Для всей выборки она равна 17,13 %. Высокое значение Wikipedia связано не с отсутствием служебных элементов на веб-странице, а с тем, что в данной лабораторной сырьём считается JSON API с уже подготовленным текстовым `extract`.

Полученные 90 документов являются исследовательской выборкой для проверки источников и правил выделения текста. Они не являются полным корпусом. Для дальнейших лабораторных массовый сбор должен выполняться поисковым роботом, а обработка — потоково, без загрузки всего корпуса в оперативную память.

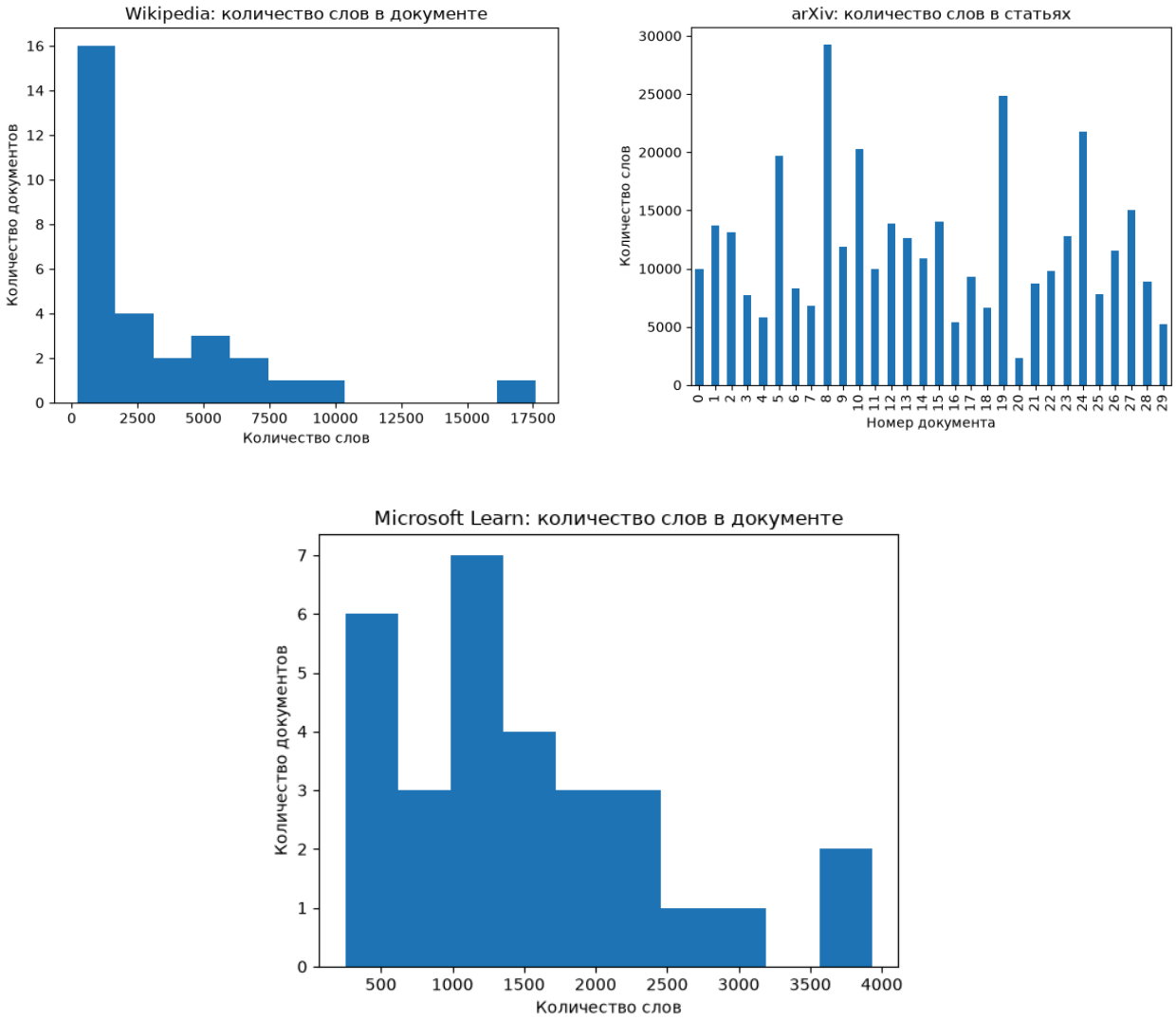


Рисунок 1 — Распределение числа слов в исследованных документах

Графики построены непосредственно в ноутбуках по полю `word_count`. Они подтверждают различие источников: arXiv содержит наиболее длинные статьи и большой разброс объёма, Wikipedia сочетает средние статьи с отдельными длинными обзорными и справочными страницами, а документы Microsoft Learn в среднем короче. Поэтому один общий порог размера для всех источников привёл бы к неравномерному отбору документов.

5.2 Проверка существующего поиска

Согласно условию задания, корпус можно использовать только при наличии уже существующего поиска. Для всех трёх источников такой поиск есть. Результаты проверки приведены в таблице 4.

Таблица 4 — Проверка существующих поисковых систем

Источник	Запрос	Наблюдение	Недостаток
Wikipedia	neural information retrieval	находятся статьи по отдельным понятиям	нет научных публикаций и документации в той же выдаче
Wikipedia	python memory model	есть результаты о Python и памяти	узкий практический вопрос плохо соответствует названиям статей
arXiv	neural information retrieval	находятся профильные научные работы	нет справочных и прикладных материалов
arXiv	vector database	есть работы о векторном поиске	часть результатов тематически близка, но не отвечает на практический вопрос
Microsoft Learn	site:learn.microsoft.com vector search HNSW	находится документация по векторным индексам	смешиваются страницы разных продуктов и версий
Microsoft Learn	site:learn.microsoft.com python async memory	находятся материалы по Python и асинхронности	неоднозначные слова относятся к нескольким продуктам

Проверялись следующие поисковые интерфейсы:

– поиск Wikipedia
– поиск arXiv
– Google с ограничением Microsoft Learn.

Основной общий недостаток состоит в разделении источников. Пользователь не может одним запросом одновременно получить энциклопедическое объяснение, научные статьи и практическую документацию. Это обосновывает создание собственного индекса по объединённому корпусу.

5.3 Выполнение требований задания

По результатам анализа выполнены все пункты задания первой лабораторной работы:

– использованы три источника при требуемых двух;

– получены локальные примеры и указаны способы их загрузки;
– описаны сырой формат, разметка и метаданные каждого источника;
– выделен основной текст и вручную проверены три конкретные страницы;
– рассчитаны количество документов, общий и средний размер сырья и текста;
– проверены существующие поисковые системы и разобраны шесть запросов;
– зафиксированы ограничения выборки и способов выделения текста.

Метрики P, DCG, NDCG и ERR здесь не вычисляются, поскольку в первой лабораторной не строится собственная ранжированная выдача и нет набора экспертных оценок релевантности. Вместо этого, как требует задание lesson-01, существующий поиск оценивается качественно по конкретным запросам. Количественная оценка ранжирования должна выполняться в отдельной лабораторной после реализации поиска.

Выборка из 90 документов нужна именно для анализа форматов. Она не заменяет полный входной корпус. Общий README курса требует не менее 30 000 документов, а для применения решения на оценку «отлично» — больше одного миллиона документов, не помещающихся целиком в оперативную память. Поэтому масштаб полного корпуса проверяется отдельно при массовом обходе источников; в этом отчёте он не подменяется размером исследовательской выборки.

5.4 Пригодность корпуса для следующих работ

Выбранные источники подходят для дальнейшей разработки поисковой системы по следующим причинам:

– у каждого документа есть устойчивый идентификатор и канонический URL;
– доступны способы массового обнаружения документов: категории MediaWiki, Atom API arXiv и sitemap Microsoft Learn;
– основной текст можно отделить от служебных блоков;
– документы содержат естественный язык, технические термины, код и формулы;

– источники имеют существующий поиск для последующего сравнения;
– разные жанры позволяют проверять поиск не только на одном типе текста.

При массовом сборе необходимо сохранить сырой документ вместе с метаданными и хешем содержимого. Тогда правила выделения текста можно менять без повторного скачивания страниц. Для исключения дублей достаточно установить уникальность пары «источник — идентификатор документа». Сами 30 документов на источник нужны для анализа, а не для ограничения размера будущего корпуса.

6 Выводы

В работе я исследовал три источника англоязычных технических документов: Wikipedia, arXiv HTML и Microsoft Learn. Для каждого источника были загружены 30 примеров, изучены формат сырого документа и метаданные, реализовано выделение основного текста и рассчитана статистика.

Суммарный объём 90 сырых документов составил 18 522 831 байт, а объём выделенного текста — 3 173 576 байт. Наибольший средний объём текста получен у arXiv — 74 764,77 байта и 11 931,97 слова на документ. Wikipedia в выбранном API-представлении содержит минимальную долю служебных данных. Microsoft Learn даёт наиболее короткие документы, но содержит полезные команды, таблицы и названия программных компонентов.

Единого правила выделения текста оказалось недостаточно. Для Wikipedia текст берётся из API-поля `extract`, для arXiv выбирается узел `article`, для Microsoft Learn — `main` с дополнительным удалением служебных блоков. Отдельные правила проще проверять и изменять при обновлении разметки сайтов.

Существующий поиск доступен для всех источников, поэтому условие задания выполнено. При этом ни одна из проверенных систем не объединяет все три вида документов. Собственная поисковая система сможет применять одинаковую токенизацию и язык запросов ко всему корпусу.

К недостаткам работы относятся небольшой размер исследовательской выборки, зависимость HTML-селекторов от разметки сайтов, линейное

представление формул arXiv и отличие API-представления Wikipedia от HTML, который будет сохраняться поисковым роботом. Эти ограничения можно уменьшить массовым сбором документов, контролем минимального размера текста, хранением исходного HTML и регулярными автотестами выделения текста на фиксированных страницах.

7 Использованные источники

1. Задание лабораторной работы № 1:

https://github.com/toshunster/MAI-IR/blob/main/lesson-01/homework.md.

2. Общие требования курса:

https://github.com/toshunster/MAI-IR/blob/main/README.md.

3. Документация MediaWiki API:

https://www.mediawiki.org/wiki/API:Main_page.

4. Руководство пользователя arXiv API:

https://info.arxiv.org/help/api/user-manual.html.

5. Индекс sitemap Microsoft Learn:

https://learn.microsoft.com/_sitemaps/sitemapindex.xml.

6. Исходный код работы:

https://github.com/dmitriikuleshov/mai-ir.